

COM0418_ImageProcessing_Week2_Lab_Instructor_Directive_Practice

September 22, 2026

1 COM0418 Image Processing - Week 2 Laboratory

1.1 NumPy Arrays, MATLAB Test Images, OpenCV I/O, Data Types, Display, and Bit-Plane Steganography

Environment: Anaconda Python / `imagepr_env` / JupyterLab

Main libraries: NumPy, OpenCV, Matplotlib, Pillow

This notebook combines and updates the earlier **Lab 2 (NumPy arrays)** and **Lab 3 (image reading)** material. All image-processing examples now use the **MATLAB test images and MATLAB image-data directory used in the original laboratory material**.

1.1.1 MATLAB test images used in this laboratory

- `cameraman.tif` - grayscale image; also used for binary images, data-type conversion, bit planes, and LSB steganography
- `baby.jpg` - true-color image read with OpenCV
- `sherlock.jpg` - true-color image read with Pillow for comparison
- `trees.tif` - indexed/palette color image

1.1.2 Learning outcomes

By the end of this laboratory, you should be able to:

1. create, reshape, index, slice, copy, and modify NumPy arrays;
2. interpret an image as a NumPy array;
3. read **binary, grayscale, true-color/RGB, and indexed-color** images;
4. explain the **BGR vs RGB** issue in OpenCV and Matplotlib;
5. inspect and convert image data types safely;
6. display and save images with Matplotlib and OpenCV;
7. extract the eight bit planes of an 8-bit grayscale image;
8. use the least significant bit (LSB) plane for a simple **steganography** demonstration.

Important: Steganography hides the existence of a message; it is not encryption. Use these techniques only on data you are permitted to modify.

Instructor format: each worked solution is followed by the exact directive cell that appears in the student notebook. These practice cells are intentionally left without solution code.

1.2 0. Laboratory setup and MATLAB image directory

The course Anaconda environment has already been created. Start JupyterLab from the activated environment:

```
conda activate imagepr_env
jupyter lab
```

The original laboratory files use MATLAB sample images located at:

C:\Program Files\MATLAB\R2023b\toolbox\images\imdata

The notebook below uses this directory as its **primary source**. If your laboratory computer has a different MATLAB release, change only the `MATLAB_IMAGE_DIR` line, for example R2024b or R2025a.

For the instructor PDF generation only, a local fallback directory containing copies reconstructed from the original Lab 3 outputs is used when MATLAB is not installed. On the laboratory PCs, the real MATLAB files should be found first.

```
[1]: # Core libraries used throughout the lab
from pathlib import Path
import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

%matplotlib inline

np.set_printoptions(linewidth=120)
print('OpenCV version :', cv2.__version__)
print('NumPy version  :', np.__version__)
```

OpenCV version : 4.13.0

NumPy version : 2.3.5

1.2.1 MATLAB test-image paths

The filenames below are taken directly from the earlier laboratory examples. A small helper function first checks the MATLAB installation directory and then uses the local fallback folder only when the MATLAB directory is unavailable.

This keeps the laboratory code close to the original MATLAB-based course material while allowing the instructor notebook to be rendered on a non-MATLAB computer.

```
[2]: MATLAB_IMAGE_DIR = Path(r'C:\Program Files\MATLAB\R2023b\toolbox\images\imdata')
LOCAL_FALLBACK_DIR = Path('matlab_test_images')

MATLAB_TEST_IMAGES = {
    'gray': 'cameraman.tif',
    'color_cv': 'baby.jpg',
    'color_pil': 'sherlock.jpg',
    'indexed': 'trees.tif',
```

```

}

def matlab_image_path(filename):
    """Return the real MATLAB image path when it exists; otherwise use fallback.
    ↪"""
    real_path = MATLAB_IMAGE_DIR / filename
    if real_path.exists():
        return real_path
    fallback_path = LOCAL_FALLBACK_DIR / filename
    if fallback_path.exists():
        return fallback_path
    raise FileNotFoundError(
        f'{filename} was not found. Check MATLAB_IMAGE_DIR: {MATLAB_IMAGE_DIR}'
    )

for key, filename in MATLAB_TEST_IMAGES.items():
    print(f'{key:10s}: {matlab_image_path(filename)}')

```

```

gray      : matlab_test_images/cameraman.tif
color_cv  : matlab_test_images/baby.jpg
color_pil : matlab_test_images/sherlock.jpg
indexed   : matlab_test_images/trees.tif

```

2 1. NumPy arrays for image processing

Digital images are stored and processed as arrays. In Python, OpenCV images are normally **NumPy ndarray objects**. Therefore, NumPy indexing and arithmetic are fundamental image-processing skills.

2.1 1.1 From Python lists to NumPy arrays

A Python list is a general container. A NumPy array stores values in a regular numeric structure, which allows fast vectorized operations.

```

[3]: mylist = [1, 2, 3, 4, 5]
      myarray = np.array(mylist)

      print('List type :', type(mylist))
      print('Array type:', type(myarray))
      print('Array      : ', myarray)
      print('dtype      : ', myarray.dtype)

```

```

List type : <class 'list'>
Array type: <class 'numpy.ndarray'>
Array      : [1 2 3 4 5]
dtype      : int64

```

Student Exercise 01 - practice cell

The following code cell is intentionally left without solution code. Students receive the same

directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Create the Python list `mylist = [1, 2, 3, 4, 5]`.
# 2. Convert the list to a NumPy array named `myarray` with `np.array`.
# 3. Print the Python type of both objects, the contents of `myarray`, and
    ↪ `myarray.dtype`.
# 4. In one sentence, note why NumPy arrays are more suitable than lists for
    ↪ image matrices.
#
# Write your code below this line.
```

2.2 1.2 Array creation, shape, dimension, size, and reshape

The earlier NumPy lab used `arange`, `zeros`, `ones`, random arrays, and `reshape`. These operations map directly to creating masks, kernels, coordinate grids, and test images.

```
[4]: a = np.arange(0, 10)
b = np.arange(0, 10, 2)
z = np.zeros((3, 5), dtype=np.uint8)
o = np.ones((2, 4), dtype=np.float32)

np.random.seed(99)
r = np.random.randint(0, 100, 10)

print('a =', a)
print('b =', b)
print('zeros shape/dtype:', z.shape, z.dtype)
print('ones shape/dtype :', o.shape, o.dtype)
print('random          : ', r)
print('min/max/mean    : ', r.min(), r.max(), r.mean())
print('argmin/argmax   : ', r.argmin(), r.argmax())
print('reshape 2x5:\n', r.reshape(2, 5))
```

```
a = [0 1 2 3 4 5 6 7 8 9]
b = [0 2 4 6 8]
zeros shape/dtype: (3, 5) uint8
ones shape/dtype : (2, 4) float32
random          : [ 1 35 57 40 73 82 68 69 52  1]
min/max/mean    : 1 82 47.8
argmin/argmax   : 0 5
reshape 2x5:
[[ 1 35 57 40 73]
 [82 68 69 52  1]]
```

Student Exercise 02 - practice cell

The following code cell is intentionally left without solution code. Students receive the same

directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Create `a` with integers 0-9 and `b` with the even integers 0-8 using `np.
    ↳`arange`.
# 2. Create a 3x5 `uint8` zero array named `z` and a 2x4 `float32` one array
    ↳named `o`.
# 3. Set `np.random.seed(99)` and generate 10 random integers in [0,100) as `r`.
# 4. Print shape/dtype information, minimum, maximum, mean, argmin, argmax, and
    ↳reshape `r` to 2x5.
#
# Write your code below this line.
```

2.3 1.3 Matrix indexing and slicing

For an array A, NumPy uses **zero-based indexing**:

- A[row, column] selects one element;
- A[:, c] selects a complete column;
- A[r, :] selects a complete row;
- A[r0:r1, c0:c1] selects a rectangular region.

The ending index is excluded. Thus 0:4 selects indices 0, 1, 2, and 3.

```
[5]: mat = np.arange(0, 100).reshape(10, 10)

print('mat shape:', mat.shape)
print('mat[0,1] =', mat[0, 1])
print('mat[4,6] =', mat[4, 6])
print('Second column:', mat[:, 1])
print('Third row      :', mat[2, :])
print('Top-left 4x4 block:\n', mat[0:4, 0:4])
```

```
mat shape: (10, 10)
mat[0,1] = 1
mat[4,6] = 46
Second column: [ 1 11 21 31 41 51 61 71 81 91]
Third row      : [20 21 22 23 24 25 26 27 28 29]
Top-left 4x4 block:
[[ 0  1  2  3]
 [10 11 12 13]
 [20 21 22 23]
 [30 31 32 33]]
```

Student Exercise 03 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Create `mat = np.arange(0,100).reshape(10,10)`.
# 2. Print its shape and the values at `[0,1]` and `[4,6]`.
# 3. Using NumPy slicing, print the second column, third row, and top-left 4x4
    ↪ block.
# 4. Remember that NumPy indexing starts from zero and the slice ending index
    ↪ is excluded.
#
# Write your code below this line.
```

2.4 1.4 Views, copies, and region modification

Slices are commonly used as **regions of interest (ROIs)**. A subtle NumPy behavior is that many slices are *views* of the original array. If you need an independent image or ROI, use `.copy()`.

```
[6]: base = np.arange(0, 100, dtype=np.uint8).reshape(10, 10)
roi_view = base[0:4, 0:4] # view into base
roi_copy = base[0:4, 0:4].copy() # independent copy

roi_view[:] = 0
roi_copy[:] = 200

print('base after changing roi_view:\n', base)
print('\nroi_copy after independent change:\n', roi_copy)
```

```
base after changing roi_view:
[[ 0  0  0  0  4  5  6  7  8  9]
 [ 0  0  0  0 14 15 16 17 18 19]
 [ 0  0  0  0 24 25 26 27 28 29]
 [ 0  0  0  0 34 35 36 37 38 39]
 [40 41 42 43 44 45 46 47 48 49]
 [50 51 52 53 54 55 56 57 58 59]
 [60 61 62 63 64 65 66 67 68 69]
 [70 71 72 73 74 75 76 77 78 79]
 [80 81 82 83 84 85 86 87 88 89]
 [90 91 92 93 94 95 96 97 98 99]]
```

```
roi_copy after independent change:
[[200 200 200 200]
 [200 200 200 200]
 [200 200 200 200]
 [200 200 200 200]]
```

Student Exercise 04 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Create the 10x10 `uint8` array `base` from values 0-99.
# 2. Create `roi_view = base[0:4,0:4]` and `roi_copy = base[0:4,0:4].copy()`.
# 3. Set all values of `roi_view` to 0 and all values of `roi_copy` to 200.
# 4. Print `base` and `roi_copy` and explain from the output why a slice view
    ↪and `.copy()` behave
# differently.
#
# Write your code below this line.
```

3 2. Grayscale images

The MATLAB sample file **cameraman.tif** is an 8-bit grayscale image. OpenCV can read it directly as a 2-D array by using `cv2.IMREAD_GRAYSCALE` (or the value 0).

```
[7]: gray_path = matlab_image_path('cameraman.tif')
gray = cv2.imread(str(gray_path), cv2.IMREAD_GRAYSCALE)

if gray is None:
    raise RuntimeError(f'OpenCV could not read: {gray_path}')

print('file :', gray_path)
print('type :', type(gray))
print('shape:', gray.shape)
print('dtype:', gray.dtype)
print('min/max:', gray.min(), gray.max())
```

```
file : matlab_test_images/cameraman.tif
type : <class 'numpy.ndarray'>
shape: (256, 256)
dtype: uint8
min/max: 0 255
```

Student Exercise 05 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Use `matlab_image_path('cameraman.tif')` to obtain `gray_path`.
# 2. Read the image with `cv2.imread(..., cv2.IMREAD_GRAYSCALE)` into `gray`.
# 3. Check for a failed read (`gray is None`).
# 4. Print file path, object type, shape, dtype, minimum, and maximum pixel
    ↪values.
#
```

```
# Write your code below this line.
```

3.1 2.1 Display the MATLAB cameraman.tif image with Matplotlib

For grayscale arrays, specify `cmap='gray'`. It is also good practice to fix the display range with `vmin=0, vmax=255` when comparing 8-bit images.

```
[8]: plt.figure(figsize=(5, 5))  
plt.imshow(gray, cmap='gray', vmin=0, vmax=255)  
plt.title('MATLAB cameraman.tif - 8-bit Grayscale')  
plt.axis('off')  
plt.show()
```

MATLAB cameraman.tif - 8-bit Grayscale



Student Exercise 06 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code  
#  
# Required operations:  
# 1. Display `gray` with Matplotlib in a 5x5 figure.
```

```
# 2. Use `cmap='gray'` and explicitly set the 8-bit display range with `vmin=0, vmax=255`.
# 3. Add an informative title, turn axes off, and call `plt.show()`.
#
# Write your code below this line.
```

3.2 2.2 Select and display a region of interest

The original Lab 3 example selected a region from `cameraman.tif` by NumPy slicing. Remember that Python indices start from zero and the ending index is excluded.

```
[9]: roi = gray[25:125, 75:175]

plt.figure(figsize=(4, 4))
plt.imshow(roi, cmap='gray', vmin=0, vmax=255)
plt.title('ROI: cameraman[25:125, 75:175]')
plt.axis('off')
plt.show()

print('ROI shape:', roi.shape)
```

ROI: cameraman[25:125, 75:175]



ROI shape: (100, 100)

Student Exercise 07 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Extract a 100x100 ROI from `gray` using rows 25:125 and columns 75:175;
    ↪ store it as `roi`.
# 2. Display the ROI in grayscale with the correct 0-255 range.
# 3. Print `roi.shape` to verify the selected region size.
#
# Write your code below this line.
```

4 3. Binary images from cameraman.tif

A logical binary image conceptually contains **0** and **1**. Image files and display routines often use **0** and **255** for black and white. Here we generate both representations from the MATLAB grayscale test image.

```
[10]: threshold_value = 127
_, binary255 = cv2.threshold(gray, threshold_value, 255, cv2.THRESH_BINARY)
binary01 = (binary255 > 0).astype(np.uint8)

print('Stored display values:', np.unique(binary255))
print('Logical 0/1 values    :', np.unique(binary01))

plt.figure(figsize=(5, 5))
plt.imshow(binary01, cmap='gray', vmin=0, vmax=1)
plt.title('Binary Image (0/1)')
plt.axis('off')
plt.show()
```

```
Stored display values: [  0 255]
Logical 0/1 values    : [0 1]
```

Binary Image (0/1)



Student Exercise 08 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Set `threshold_value = 127`.
# 2. Use `cv2.threshold` to generate `binary255`, whose pixels are 0 or 255.
# 3. Create `binary01` from `binary255` as a `uint8` array containing only 0
    ↪ and 1.
# 4. Print unique values of both representations and display `binary01` with
    ↪ `vmin=0, vmax=1`.
#
# Write your code below this line.
```

5 4. True-color images: OpenCV BGR vs RGB

The MATLAB test file `baby.jpg` is used for the OpenCV color example.

Critical OpenCV detail:

- `cv2.imread()` decodes a normal color image into **BGR** channel order.
- Matplotlib `plt.imshow()` expects **RGB** channel order.
- Therefore, an OpenCV BGR array displayed directly with Matplotlib has red and blue interchanged.

```
[11]: baby_path = matlab_image_path('baby.jpg')
img_bgr = cv2.imread(str(baby_path), cv2.IMREAD_COLOR)

if img_bgr is None:
    raise RuntimeError(f'OpenCV could not read: {baby_path}')

print('file :', baby_path)
print('shape:', img_bgr.shape)
print('dtype:', img_bgr.dtype)
print('Pixel [100,100] in OpenCV BGR order:', img_bgr[100, 100])
```

```
file : matlab_test_images/baby.jpg
shape: (720, 450, 3)
dtype: uint8
Pixel [100,100] in OpenCV BGR order: [239 231 194]
```

Student Exercise 09 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Use `matlab_image_path('baby.jpg')` to obtain `baby_path`.
# 2. Read it with `cv2.imread(..., cv2.IMREAD_COLOR)` into `img_bgr` and check
    ↪ for a failed read.
# 3. Print shape and dtype.
# 4. Print pixel `[100,100]` and identify that the three returned values are in
    ↪ B, G, R order.
#
# Write your code below this line.
```

5.1 4.1 Correct the band order for Matplotlib

Use `cv2.cvtColor(image, cv2.COLOR_BGR2RGB)` or reverse the last array axis with `image[:, :, ::-1]`. The comparison below makes the OpenCV/Matplotlib band-order issue visible.

```
[12]: img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
img_rgb_numpy = img_rgb[:, :, ::-1]

print('cv2.cvtColor and NumPy reversal identical:', np.array_equal(img_rgb,
    ↪ img_rgb_numpy))

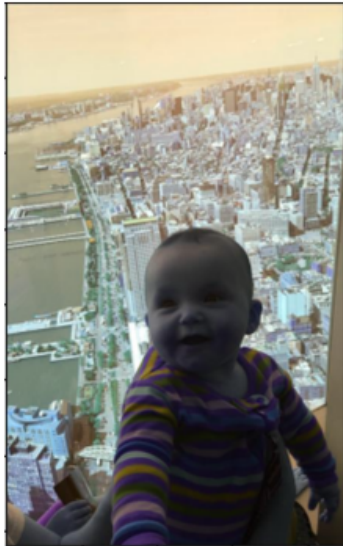
plt.figure(figsize=(10, 4))
```

```
plt.subplot(1, 2, 1)
plt.imshow(img_bgr)
plt.title('Incorrect in Matplotlib: BGR shown as RGB')
plt.axis('off')

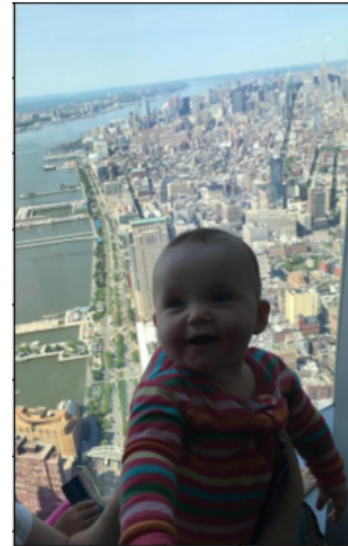
plt.subplot(1, 2, 2)
plt.imshow(img_rgb)
plt.title('Correct: BGR -> RGB')
plt.axis('off')
plt.tight_layout()
plt.show()
```

cv2.cvtColor and NumPy reversal identical: True

Incorrect in Matplotlib: BGR shown as RGB



Correct: BGR -> RGB



Student Exercise 10 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Convert `img_bgr` to `img_rgb` using `cv2.cvtColor(..., cv2.
    ↳COLOR_BGR2RGB)` .
# 2. Create a second RGB version named `img_rgb_numpy` by reversing the last
    ↳axis.
# 3. Verify with `np.array_equal` that the two conversion methods are identical.
# 4. Use two Matplotlib subplots to show the incorrect direct BGR display and
    ↳the corrected RGB
```

```
# display.
#
# Write your code below this line.
```

5.2 4.2 Inspect the RGB channels

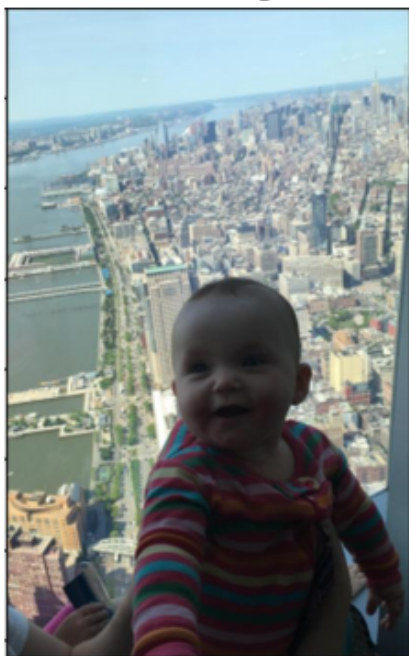
For a NumPy RGB array with shape (rows, columns, 3):

- [:,:,0] is red,
- [:,:,1] is green,
- [:,:,2] is blue.

```
[13]: R = img_rgb[:, :, 0]
      G = img_rgb[:, :, 1]
      B = img_rgb[:, :, 2]

      plt.figure(figsize=(9, 8))
      for i, (image, title) in enumerate([
          (img_rgb, 'RGB image'),
          (R, 'Red channel'),
          (G, 'Green channel'),
          (B, 'Blue channel')
      ], start=1):
          plt.subplot(2, 2, i)
          if image.ndim == 2:
              plt.imshow(image, cmap='gray', vmin=0, vmax=255)
          else:
              plt.imshow(image)
          plt.title(title)
          plt.axis('off')
      plt.tight_layout()
      plt.show()
```

RGB image



Red channel



Green channel



Blue channel



Student Exercise 11 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. From `img_rgb`, extract `R`, `G`, and `B` using channel indices 0, 1, and
    ↪ 2.
# 2. Create a 2x2 figure containing the RGB image and each channel displayed as
    ↪ a grayscale intensity
# image.
# 3. Use fixed 0-255 scaling for the individual channel images and turn axes
    ↪ off.
#
# Write your code below this line.
```

5.3 4.3 Read a MATLAB true-color test image with Pillow

The original Lab 3 also used **sherlock.jpg**. Pillow reads a normal JPEG into **RGB** order, so after conversion to a NumPy array it can be displayed directly by Matplotlib. This is different from OpenCV's default BGR order.

```
[14]: sherlock_path = matlab_image_path('sherlock.jpg')
sherlock_pil = Image.open(sherlock_path)
sherlock_rgb = np.asarray(sherlock_pil)

print('file:', sherlock_path)
print('PIL mode:', sherlock_pil.mode)
print('type:', type(sherlock_rgb))
print('shape:', sherlock_rgb.shape)
print('dtype:', sherlock_rgb.dtype)

plt.figure(figsize=(7, 4.5))
plt.imshow(sherlock_rgb)
plt.title('MATLAB sherlock.jpg read by Pillow (RGB)')
plt.axis('off')
plt.show()
```

```
file : matlab_test_images/sherlock.jpg
PIL mode: RGB
type: <class 'numpy.ndarray'>
shape: (640, 960, 3)
dtype: uint8
```

MATLAB sherlock.jpg read by Pillow (RGB)



Student Exercise 12 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Use `matlab_image_path('sherlock.jpg')` and open the file with `PIL.Image.
    ↳open` as
# `sherlock_pil`.
# 2. Convert it to the NumPy array `sherlock_rgb` with `np.asarray`.
# 3. Print the Pillow mode, NumPy type, shape, and dtype.
# 4. Display it directly with Matplotlib and explain why no BGR-to-RGB
    ↳conversion is required.
#
# Write your code below this line.
```

5.4 4.4 MATLAB vs OpenCV: reading color images

Topic	MATLAB imread	OpenCV cv2.imread
Channel order	RGB	BGR by default
Array indexing	1-based in MATLAB code	0-based in Python
Typical 8-bit type	uint8	numpy.uint8

Topic	MATLAB imread	OpenCV cv2.imread
Color shape	rows x cols x 3	rows x cols x 3
Indexed image	can return index array + colormap	normally expands palette to BGR
Alpha channel	format-dependent	use IMREAD_UNCHANGED to keep alpha

The underlying image file does not become “BGR”; OpenCV simply returns the decoded color channels in BGR order.

6 5. Indexed-color images: MATLAB `trees.tif`

The MATLAB sample file `trees.tif` is used as the indexed/palette image example. An indexed image stores an array of **palette indices** together with a **color map** rather than three color samples at every pixel.

OpenCV normally expands a palette image to a regular BGR array. Pillow is therefore more convenient when we want to inspect the palette representation itself.

```
[15]: trees_path = matlab_image_path('trees.tif')
      indexed_pil = Image.open(trees_path)

      print('file :', trees_path)
      print('PIL mode:', indexed_pil.mode)

      # If the MATLAB file is palette/indexed (mode P), this is its index array.
      # The local PDF-rendering fallback is also stored as a palette TIFF.
      index_array = np.array(indexed_pil)
      palette = indexed_pil.getpalette()

      print('Index array shape/dtype:', index_array.shape, index_array.dtype)
      print('Number of used index values:', np.unique(index_array).size)
      print('Palette available:', palette is not None)
      if palette is not None:
          print('First 18 palette values (R,G,B triplets):', palette[:18])
```

```
file : matlab_test_images/trees.tif
PIL mode: P
Index array shape/dtype: (258, 350) uint8
Number of used index values: 64
Palette available: True
First 18 palette values (R,G,B triplets): [255, 255, 255, 255, 255, 253, 253,
255, 255, 248, 252, 253, 246, 244, 235, 223, 243, 255]
```

Student Exercise 13 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Open MATLAB `trees.tif` with Pillow as `indexed_pil`.
# 2. Print its Pillow mode and convert the stored pixel indices to
    ↪ `index_array` with `np.array`.
# 3. Read its palette using `getpalette()` and store it as `palette`.
# 4. Print the index-array shape/dtype, number of used index values, whether a
    ↪ palette exists, and the
# first six RGB triplets when available.
#
# Write your code below this line.
```

6.1 5.1 Display the indexed image correctly

Convert the palette image to RGB before passing it to Matplotlib.

```
[16]: indexed_rgb = np.array(indexed_pil.convert('RGB'))

plt.figure(figsize=(6, 4.5))
plt.imshow(indexed_rgb)
plt.title('MATLAB trees.tif - Indexed Image Converted to RGB')
plt.axis('off')
plt.show()
```

MATLAB trees.tif - Indexed Image Converted to RGB



Student Exercise 14 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Convert the palette image `indexed_pil` to RGB and then to NumPy array
#    ↳ `indexed_rgb`.
# 2. Display the correctly converted `trees.tif` image using Matplotlib with an
#    ↳ informative title and
# no axes.
#
# Write your code below this line.
```

6.2 5.2 What happens if OpenCV reads an indexed image?

OpenCV does not normally return the palette indices. It expands the palette into a 3-channel BGR image.

```
[17]: indexed_cv = cv2.imread(str(trees_path), cv2.IMREAD_COLOR)
print('OpenCV result shape:', indexed_cv.shape)
print('OpenCV result dtype:', indexed_cv.dtype)
print('OpenCV returns a decoded 3-channel BGR image rather than palette indices.
      ↳')
```

OpenCV result shape: (258, 350, 3)

OpenCV result dtype: uint8

OpenCV returns a decoded 3-channel BGR image rather than palette indices.

Student Exercise 15 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Read `trees_path` with OpenCV using `cv2.IMREAD_COLOR` into `indexed_cv`.
# 2. Print its shape and dtype.
# 3. Compare this result with Pillow's index array and state what OpenCV
#    ↳ returns instead of palette
# indices.
#
# Write your code below this line.
```

7 6. Image data types and safe conversions

The array data type affects storage, arithmetic, and display.

Common image types:

- uint8: 0...255, one byte per channel;
- uint16: 0...65535, two bytes per channel;
- float32 / float64: often used for processing; values may be normalized to 0...1.

Always inspect `shape`, `dtype`, and the numeric range before processing an unfamiliar image.

```
[18]: def describe(name, image):  
        print(f'{name:18s} shape={image.shape}, dtype={image.dtype}, min={image.  
        ↪min()}, max={image.max()}')  
  
describe('gray', gray)  
describe('img_bgr', img_bgr)
```

```
gray                shape=(256, 256), dtype=uint8, min=0, max=255  
img_bgr             shape=(720, 450, 3), dtype=uint8, min=0, max=255
```

Student Exercise 16 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code  
#  
# Required operations:  
# 1. Define a helper function `describe(name, image)` that prints shape, dtype,   
    ↪minimum, and maximum  
# values.  
# 2. Call the function for `gray` and `img_bgr`.  
# 3. Use this function later whenever you need to verify the numeric   
    ↪representation of an image.  
#  
# Write your code below this line.
```

7.1 6.1 Convert uint8 to floating point correctly

`astype(np.float32)` changes the type but **does not automatically scale the numeric values**. For normalized floating-point images, divide by 255.

```
[19]: gray_float_raw = gray.astype(np.float32)  
gray_float01 = gray.astype(np.float32) / 255.0  
  
print('Raw float range      :', gray_float_raw.min(), gray_float_raw.max())  
print('Normalized float range:', gray_float01.min(), gray_float01.max())  
  
plt.figure(figsize=(8, 3))  
plt.subplot(1, 2, 1)  
plt.imshow(gray_float_raw, cmap='gray', vmin=0, vmax=255)  
plt.title('float32, values 0...255')  
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
plt.imshow(gray_float01, cmap='gray', vmin=0, vmax=1)
plt.title('float32, normalized 0...1')
plt.axis('off')
plt.tight_layout()
plt.show()
```

Raw float range : 0.0 255.0

Normalized float range: 0.0 1.0

float32, values 0...255



float32, normalized 0...1



Student Exercise 17 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Convert `gray` to `float32` without scaling as `gray_float_raw`.
# 2. Create normalized `gray_float01` by converting to `float32` and dividing
    ↪ by 255.0.
# 3. Print the numeric ranges of both arrays.
# 4. Display them side by side with matching display ranges (0-255 and 0-1) to
    ↪ show that dtype
# conversion and normalization are different operations.
#
# Write your code below this line.
```

7.2 6.2 Convert normalized floating point back to uint8

Use **scaling + rounding + clipping + conversion**. Direct `astype(np.uint8)` does not protect you from values below 0 or above 255.

```
[20]: gray_back = np.clip(np.round(gray_float01 * 255.0), 0, 255).astype(np.uint8)
print('Exact reconstruction:', np.array_equal(gray, gray_back))
```

Exact reconstruction: True

Student Exercise 18 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Convert normalized `gray_float01` back to `uint8` as `gray_back`.
# 2. Use multiply-by-255, rounding, clipping to [0,255], and `astype(np.uint8)`.
# 3. Use `np.array_equal` to verify whether the original `gray` image is
    ↪reconstructed exactly.
#
# Write your code below this line.
```

7.3 6.3 Convert the MATLAB grayscale image to 16-bit

The attached laboratory examples identify the MATLAB test-image directory and filenames but do not provide a separate 16-bit MATLAB sample filename. Therefore, we demonstrate **data-type conversion** using `cameraman.tif` itself.

A meaningful 8-bit-to-16-bit conversion can map $0 \dots 255$ to $0 \dots 65535$. Multiplying by 257 performs this full-range mapping because $255 \times 257 = 65535$.

```
[21]: gray_u16 = gray.astype(np.uint16) * 257

describe('gray uint8', gray)
describe('gray uint16', gray_u16)

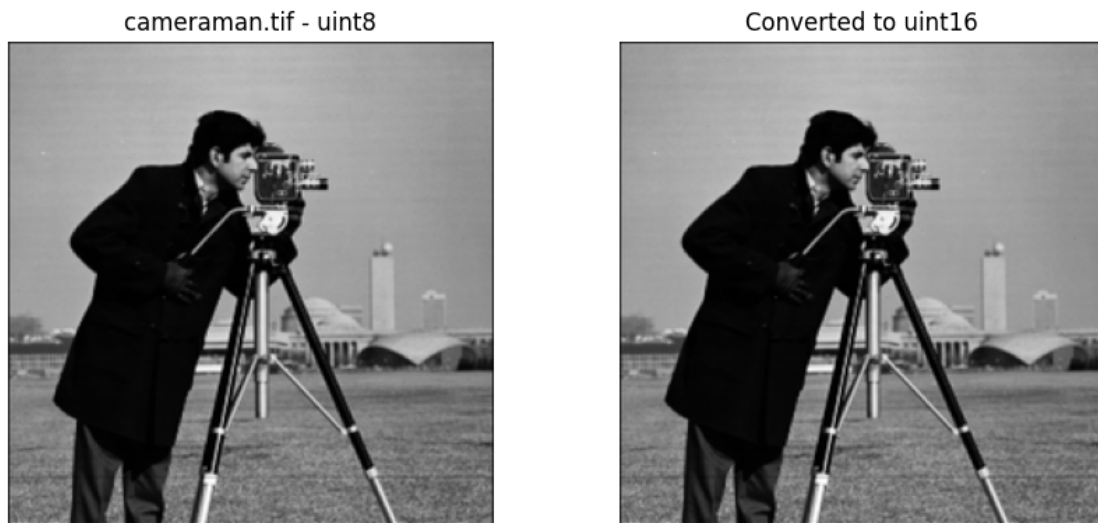
# Map back to 8-bit and verify that the original values are recovered.
gray_from_u16 = np.round(gray_u16.astype(np.float32) / 257.0).astype(np.uint8)
print('Recovered exactly:', np.array_equal(gray, gray_from_u16))

plt.figure(figsize=(9, 4))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray', vmin=0, vmax=255)
plt.title('cameraman.tif - uint8')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(gray_u16, cmap='gray', vmin=0, vmax=65535)
plt.title('Converted to uint16')
plt.axis('off')
plt.tight_layout()
plt.show()
```

```

gray uint8          shape=(256, 256), dtype=uint8, min=0, max=255
gray uint16         shape=(256, 256), dtype=uint16, min=0, max=65535
Recovered exactly: True

```



Student Exercise 19 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```

[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Convert `gray` to `uint16` as `gray_u16` and map the full 8-bit range to
    ↳ 16-bit by multiplying by
# 257.
# 2. Use `describe` to compare `gray` and `gray_u16`.
# 3. Convert back to 8-bit as `gray_from_u16` and verify exact recovery.
# 4. Display the 8-bit and 16-bit versions side by side using their appropriate
    ↳ intensity ranges.
#
# Write your code below this line.

```

7.4 6.4 NumPy overflow vs OpenCV saturated arithmetic

This distinction is important in image processing. `uint8` NumPy arithmetic can wrap around modulo 256, while functions such as `cv2.add` use saturation for 8-bit images.

Implementation note: for the scalar demonstration below, a `1 x 1` image array is used so `cv2.add` applies normal saturated image arithmetic.

```
[ ]: p = np.array([[250]], dtype=np.uint8)
with np.errstate(over='ignore'):
    numpy_result = p + np.uint8(20)

# Use equally shaped 2-D uint8 arrays so OpenCV performs saturated image
↳arithmetic.
opencv_result = cv2.add(p, np.array([[20]], dtype=np.uint8))

print('NumPy uint8 250 + 20 ->', int(numpy_result[0, 0]), '(wrap-around)')
print('cv2.add 250 + 20 ->', int(opencv_result[0, 0]), '(saturated at 255)')
```

NumPy uint8 250 + 20 -> 14 (wrap-around)
cv2.add 250 + 20 -> 255 (saturated at 255)

Student Exercise 20 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Create a 1x1 `uint8` image `p` containing 250.
# 2. Add 20 with ordinary NumPy `uint8` arithmetic and store the result as
↳`numpy_result`.
# 3. Add 20 with `cv2.add` using an equally shaped `uint8` array and store it
↳as `opencv_result`.
# 4. Print both results and explain wrap-around (NumPy) versus saturation at
↳255 (OpenCV).
#
# Write your code below this line.
```

8 7. Displaying images in JupyterLab

8.1 7.1 Matplotlib - recommended for notebooks

Matplotlib works inline, supports subplots, titles, axes, and colormaps, and is therefore convenient for laboratory reports.

Remember:

- grayscale: `plt.imshow(gray, cmap='gray');`
- OpenCV color: convert BGR to RGB first;
- use `plt.axis('off')` for a clean image-only view.

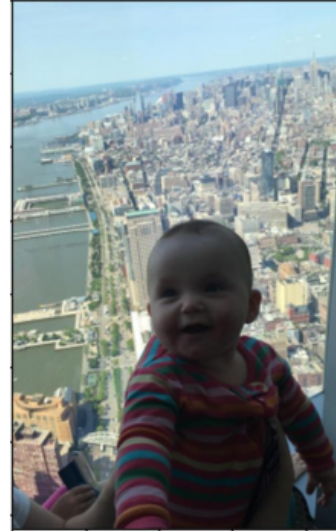
```
[23]: plt.figure(figsize=(9, 4))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray', vmin=0, vmax=255)
plt.title('Matplotlib: grayscale')
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
plt.title('Matplotlib: OpenCV BGR -> RGB')
plt.axis('off')
plt.tight_layout()
plt.show()
```

Matplotlib: grayscale



Matplotlib: OpenCV BGR -> RGB



Student Exercise 21 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Create a two-panel Matplotlib figure.
# 2. Display `gray` correctly as grayscale in the first panel.
# 3. Display `img_bgr` correctly in the second panel by converting BGR to RGB
    ↪ before `plt.imshow`.
# 4. Add titles, remove axes, and use `tight_layout`.
#
# Write your code below this line.
```

8.2 7.2 OpenCV GUI display (cv2.imshow)

On a local desktop Python installation, OpenCV can open a separate GUI window. In JupyterLab this may depend on the operating system and GUI backend, so Matplotlib is usually more reliable inside the notebook.

The code below is intentionally disabled during automatic execution. Change `RUN_OPENCV_GUI` to `True` if you want to test it on a lab PC.

```
[24]: RUN_OPENCV_GUI = False

if RUN_OPENCV_GUI:
    cv2.imshow('OpenCV BGR image', img_bgr)
    cv2.waitKey(0)           # wait for a key
    cv2.destroyAllWindows()  # close OpenCV windows
```

Student Exercise 22 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Set `RUN_OPENCV_GUI = False` initially.
# 2. Inside an `if RUN_OPENCV_GUI:` block, show `img_bgr` with `cv2.imshow`,
    ↪ wait for a key with
# `cv2.waitKey(0)`, and close windows with `cv2.destroyAllWindows()`.
# 3. Only change the flag to True on a lab PC where GUI windows are supported.
#
# Write your code below this line.
```

8.3 7.3 Writing images with OpenCV

`cv2.imwrite()` expects the array in OpenCV channel order. If your working image is RGB, convert it back to BGR before writing with OpenCV.

```
[25]: OUTPUT_DIR = Path('week2_lab_output')
OUTPUT_DIR.mkdir(exist_ok=True)

out_bgr = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2BGR)
out_path = OUTPUT_DIR / 'baby_written_by_opencv.png'
ok = cv2.imwrite(str(out_path), out_bgr)
print('Write successful:', ok, '|', out_path)
```

Write successful: True | week2_lab_output/baby_written_by_opencv.png

Student Exercise 23 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Create output folder `week2_lab_output` using `Path` and
    ↪ `mkdir(exist_ok=True)`; store it as
```

```
# `OUTPUT_DIR`.
# 2. Convert `img_rgb` back to BGR as `out_bgr` because `cv2.imwrite` expects
↳ OpenCV channel order.
# 3. Write the image as `baby_written_by_opencv.png` and print whether the
↳ write succeeded.
#
# Write your code below this line.
```

9 8. Bit planes of MATLAB cameraman.tif

Each 8-bit pixel can be written as:

$$p = b_7 \cdot 2^7 + b_6 \cdot 2^6 + \dots + b_1 \cdot 2^1 + b_0 \cdot 2^0$$

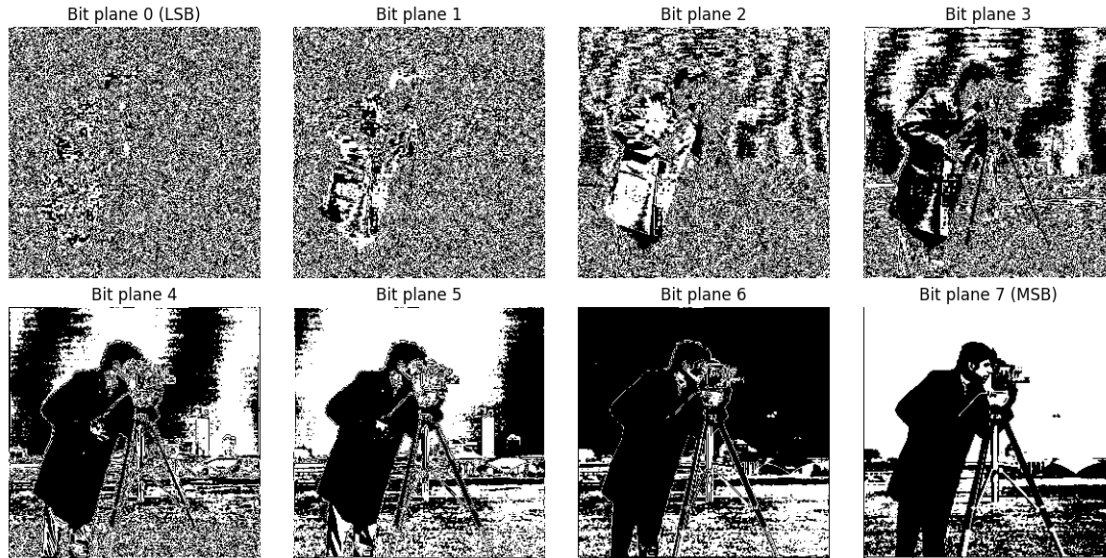
where each b_k is 0 or 1.

- **Bit 7** is the most significant bit (MSB).
- **Bit 0** is the least significant bit (LSB).

The upper bit planes contain much of the visually important structure. The lower planes often look noise-like, which is why the LSB can be used for simple data hiding.

```
[26]: bit_planes = [(gray >> k) & 1).astype(np.uint8) for k in range(8)]

plt.figure(figsize=(12, 6))
for k in range(8):
    plt.subplot(2, 4, k + 1)
    plt.imshow(bit_planes[k], cmap='gray', vmin=0, vmax=1)
    plt.title(f'Bit plane {k}' + (' (LSB)' if k == 0 else '') + (' (MSB)' if k
↳ == 7 else ''))
    plt.axis('off')
plt.tight_layout()
plt.show()
```



Student Exercise 24 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Extract all eight bit planes of the 8-bit grayscale image `gray` using bit_
    ↳ shifts and bitwise
# AND; store them in list `bit_planes`.
# 2. Display planes 0-7 in a 2x4 layout using grayscale display with range 0-1.
# 3. Label bit 0 as LSB and bit 7 as MSB, then compare how much visual_
    ↳ structure appears in low- and
# high-order planes.
#
# Write your code below this line.
```

9.1 8.1 Reconstruct the image from selected bit planes

A bit plane contributes its binary values multiplied by 2^k . We can reconstruct the image from all planes or intentionally keep only the most significant planes.

```
[27]: reconstructed = np.zeros_like(gray, dtype=np.uint16)
for k, plane in enumerate(bit_planes):
    reconstructed += plane.astype(np.uint16) * (2 ** k)
reconstructed = reconstructed.astype(np.uint8)

# Keep only the four most significant bits (planes 4,5,6,7)
msb4 = (gray & 0b11110000).astype(np.uint8)
```

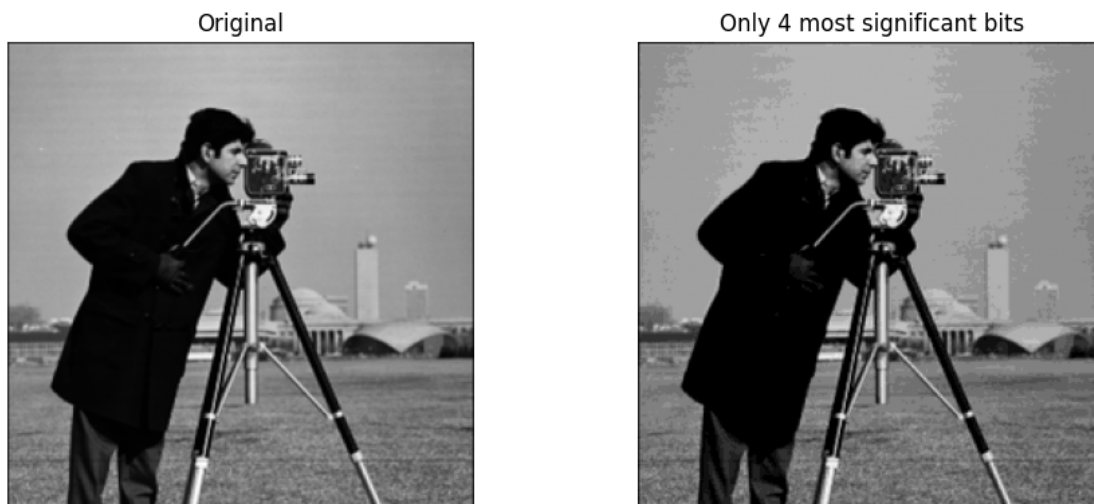
```

print('All-plane reconstruction exact:', np.array_equal(gray, reconstructed))

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray', vmin=0, vmax=255)
plt.title('Original')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(msb4, cmap='gray', vmin=0, vmax=255)
plt.title('Only 4 most significant bits')
plt.axis('off')
plt.tight_layout()
plt.show()

```

All-plane reconstruction exact: True



Student Exercise 25 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```

[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Reconstruct the image from all values in `bit_planes` by multiplying each
#    plane by  $2^k$ ; store
#    the result as `reconstructed`.
# 2. Create `msb4` by keeping only bits 4-7 of `gray` using mask `0b11110000`.
# 3. Verify that all-plane reconstruction equals `gray` exactly.
# 4. Display the original and the four-MSB reconstruction side by side.

```

```
#  
# Write your code below this line.
```

10 9. LSB steganography using MATLAB cameraman.tif

We will hide a short text message in the **least significant bit** of a grayscale image.

The procedure is:

1. encode the text as bytes;
2. prepend a 32-bit message-length header;
3. convert the bytes to a bit stream;
4. clear the LSB of selected pixels;
5. write the message bits into those LSBs;
6. reverse the process to recover the text.

Only the pixel values that carry a message bit can change, and each change is at most 1 intensity level.

File-format warning: Save LSB steganography results in a **lossless** format such as PNG. JPEG compression can alter LSB values and destroy the hidden data.

```
[28]: def text_to_bits(text: str) -> np.ndarray:  
    payload = text.encode('utf-8')  
    length_header = len(payload).to_bytes(4, byteorder='big') # 32-bit byte_  
    ↪length  
    data_bytes = length_header + payload  
    return np.unpackbits(np.frombuffer(data_bytes, dtype=np.uint8))  
  
def embed_lsb(gray_u8: np.ndarray, text: str) -> np.ndarray:  
    if gray_u8.dtype != np.uint8 or gray_u8.ndim != 2:  
        raise ValueError('embed_lsb expects a 2-D uint8 grayscale image')  
    bits = text_to_bits(text)  
    flat = gray_u8.flatten().copy()  
    if bits.size > flat.size:  
        raise ValueError(f'Message needs {bits.size} pixels but image has {flat.  
    ↪size}.')  
    flat[:bits.size] = (flat[:bits.size] & 0xFE) | bits  
    return flat.reshape(gray_u8.shape)  
  
def extract_lsb(stego_u8: np.ndarray) -> str:  
    flat_bits = (stego_u8.flatten() & 1).astype(np.uint8)  
    header = np.packbits(flat_bits[:32]).tobytes()  
    n_bytes = int.from_bytes(header, byteorder='big')  
    end = 32 + n_bytes * 8  
    payload = np.packbits(flat_bits[32:end]).tobytes()  
    return payload.decode('utf-8')
```

Student Exercise 26 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Define `text_to_bits(text)` to UTF-8 encode text, prepend a 4-byte
    ↳big-endian length header, and
# return an unpacked bit array.
# 2. Define `embed_lsb(gray_u8, text)` to validate a 2-D uint8 image, clear the
    ↳required LSBs, insert
# the payload bits, and return the stego image.
# 3. Define `extract_lsb(stego_u8)` to read the 32-bit length header, recover
    ↳the payload bits, repack
# bytes, and decode UTF-8 text.
# 4. Include a capacity check before embedding so messages that are too long
    ↳raise a clear error.
#
# Write your code below this line.
```

```
[29]: secret_message = 'COM0418 - Image Processing Lab: LSB steganography works!'
stego = embed_lsb(gray, secret_message)
recovered_message = extract_lsb(stego)

print('Original message :', secret_message)
print('Recovered message:', recovered_message)
print('Messages equal   :', secret_message == recovered_message)
```

```
Original message : COM0418 - Image Processing Lab: LSB steganography works!
Recovered message: COM0418 - Image Processing Lab: LSB steganography works!
Messages equal   : True
```

Student Exercise 27 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Create a short string `secret_message`.
# 2. Call `embed_lsb(gray, secret_message)` to create `stego`.
# 3. Recover the text with `extract_lsb(stego)` into `recovered_message`.
# 4. Print original/recovered messages and verify they are equal.
#
# Write your code below this line.
```

10.1 9.1 Measure and visualize the change

Because only the LSB is modified, the absolute difference should be 0 or 1. The displayed difference is multiplied by 255 only to make changed pixels visible.

```
[30]: diff = cv2.absdiff(gray, stego)
      changed = np.count_nonzero(diff)

      mse = np.mean((gray.astype(np.float32) - stego.astype(np.float32)) ** 2)
      psnr = np.inf if mse == 0 else 10 * np.log10((255.0 ** 2) / mse)

      print('Changed pixels:', changed, 'of', gray.size)
      print('Maximum absolute change:', diff.max())
      print('MSE:', mse)
      print('PSNR:', psnr, 'dB')

      plt.figure(figsize=(12, 4))
      plt.subplot(1, 3, 1)
      plt.imshow(gray, cmap='gray', vmin=0, vmax=255)
      plt.title('Original')
      plt.axis('off')
      plt.subplot(1, 3, 2)
      plt.imshow(stego, cmap='gray', vmin=0, vmax=255)
      plt.title('Stego image')
      plt.axis('off')
      plt.subplot(1, 3, 3)
      plt.imshow(diff * 255, cmap='gray', vmin=0, vmax=255)
      plt.title('Changed LSB locations')
      plt.axis('off')
      plt.tight_layout()
      plt.show()
```

Changed pixels: 231 of 65536

Maximum absolute change: 1

MSE: 0.0035247803

PSNR: 72.659485 dB



Student Exercise 28 - practice cell

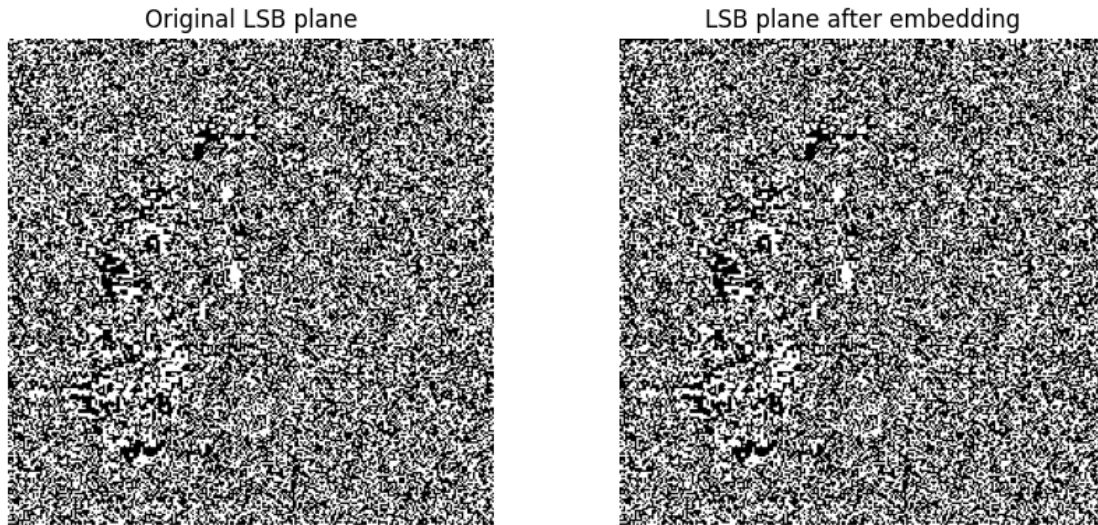
The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Compute absolute pixel difference `diff` between `gray` and `stego` with
    ↪ `cv2.absdiff`.
# 2. Count changed pixels, compute MSE, and compute PSNR for 8-bit images.
# 3. Print changed-pixel count, maximum absolute change, MSE, and PSNR.
# 4. Display original, stego, and an amplified (`diff * 255`) map of changed
    ↪ LSB locations in a 1x3
# figure.
#
# Write your code below this line.
```

10.2 9.2 Compare the LSB planes before and after embedding

```
[31]: lsb_original = gray & 1
lsb_stego = stego & 1

plt.figure(figsize=(9, 4))
plt.subplot(1, 2, 1)
plt.imshow(lsb_original, cmap='gray', vmin=0, vmax=1)
plt.title('Original LSB plane')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(lsb_stego, cmap='gray', vmin=0, vmax=1)
plt.title('LSB plane after embedding')
plt.axis('off')
plt.tight_layout()
plt.show()
```



Student Exercise 29 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Extract the original and stego LSB planes as `lsb_original = gray & 1` and
#    ↪ `lsb_stego = stego & 1`
# 2. Display both LSB planes side by side with grayscale range 0-1.
# 3. Observe whether the text embedding is visually obvious from the LSB-plane
#    ↪ images.
#
# Write your code below this line.
```

10.3 9.3 Save and reload the stego image

PNG is lossless, so the hidden message survives a normal save/read cycle.

```
[32]: stego_path = OUTPUT_DIR / 'cameraman_stego.png'
cv2.imwrite(str(stego_path), stego)
stego_reloaded = cv2.imread(str(stego_path), cv2.IMREAD_GRAYSCALE)

print('Saved to:', stego_path)
print('Reloaded message:', extract_lsb(stego_reloaded))
```

Saved to: week2_lab_output/cameraman_stego.png

Reloaded message: COM0418 - Image Processing Lab: LSB steganography works!

Student Exercise 30 - practice cell

The following code cell is intentionally left without solution code. Students receive the same directives in their notebook.

```
[ ]: # STUDENT PRACTICE CELL - intentionally left without solution code
#
# Required operations:
# 1. Save `stego` losslessly as `cameraman_stego.png` inside `OUTPUT_DIR` using
    ↪ `cv2.imwrite`.
# 2. Reload the file as grayscale into `stego_reloaded`.
# 3. Use `extract_lsb(stego_reloaded)` and print the recovered text to verify
    ↪ that PNG preserved the
# hidden bits.
#
# Write your code below this line.
```

11 10. Laboratory review and extension exercises

Before leaving the laboratory, make sure you can explain each of the following:

1. Why is `baby.jpg` displayed with wrong colors if the OpenCV BGR array is passed directly to `plt.imshow`?
2. What is the difference between a grayscale image and the thresholded binary version of `cameraman.tif`?
3. Why is Pillow more useful than ordinary `cv2.imread` when examining the index array and palette of `trees.tif`?
4. What is the difference between changing a NumPy data type and changing the numeric range?
5. Why can `uint8` NumPy arithmetic overflow or wrap around, while `cv2.add` saturates?
6. Which bit planes of `cameraman.tif` contain the strongest visible structure?
7. Why should an LSB stego image be stored in PNG rather than JPEG?
8. What channel-order difference exists between OpenCV (`baby.jpg`) and Pillow (`sherlock.jpg`)?

11.0.1 Suggested extra work

- Change the threshold used to create the binary version of `cameraman.tif` and observe the result.
- Display `baby.jpg` once without BGR-to-RGB conversion and once with the correct conversion.
- Inspect a small region of the `trees.tif` index array and identify the corresponding palette colors.
- Reconstruct `cameraman.tif` using only bit planes 5, 6, and 7.
- Hide a different short message in the LSB of `cameraman.tif` and compare the number of changed pixels.
- Try writing the stego image as JPEG and test whether the hidden message survives.